

PilotFish XSLT Transformation Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This is the workhorse **XML transformer**: it runs an XSLT stylesheet against the transaction body and replaces the stream with the transform output. Choose Xalan (XSLT 1.0) or Saxon (XSLT 3.1), pass parameters, and optionally cache compiled templates for speed.

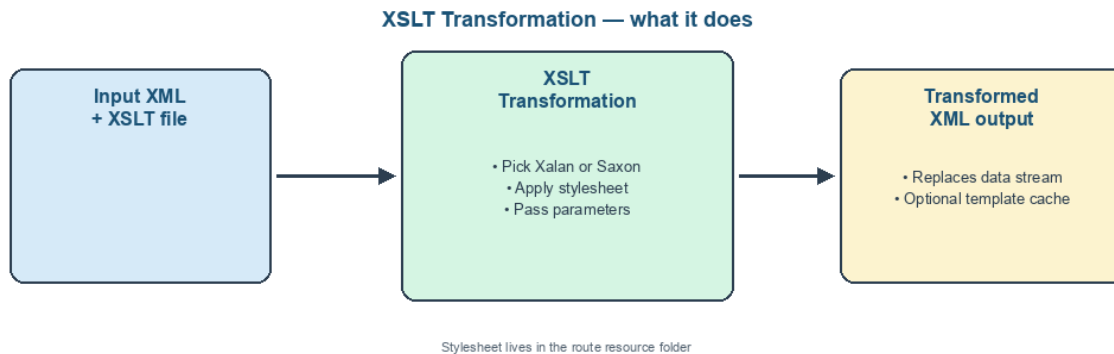


Figure 1: At a glance (plain English)

- **Loads** an XSL file from the route resource folder.
- **Transforms** the current transaction XML through the selected engine.
- **Replaces** the data stream with the result.
- **Supports** XSLT parameters and Saxon compatibility options.

Purpose

Deep dive into **XSLT Transformation** (XSLTProcessor) in `eip.war.hs.26R1.11`.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-transform-1.0.1.jar</code>
Core utilities	<code>xcs-core-1.2.6.jar</code> (XSLTTransformer, XsltConfigTypes)
Implementation class	<code>com.pilotfish.eip.modules.transform.XSLTPr</code>
Type string	XSLT Transformation
Help ID	<code>console.processor.xslt_transformation</code>
Callback type	XSLT_PROCESSOR
Categories	XML, PILOT_FISH

1. Conceptual model

Transaction XML stream + XSLT file (route resources)

```
|
v
XSLTProcessor -> XSLTTransformer.xsltTransform(...)
|
+-- engine: Xalan Interpreted | Xalan Compiled | Saxon 3.1
+-- optional template cache
+-- XSLTParameters (name/value, expression-evaluated)
+-- Saxon converter handling (compatibility tab)
|
v
data.setDataStream(resultStream)
```

2. High-level behavior (processData)

1. Resolve **XSL File** (XSLTPath) via `manager.getFile(..., loader)`.
2. If missing or unreadable -> `EIException` with absolute path in message.
3. Build XSLTTransformer; read:
 - `CacheXSLTToXML` (boolean)
 - `XSLTEngine` -> engine class via `XSLTTransformer.getEngineClass(...)`
 - `SaxonConverterHandling`, `SaxonConverterEncoding`
 - **XSLT Parameters** table -> `HashMap` with expression-evaluated keys/values
4. `xsltTransformer.xsltTransform(xsltFile, StreamSource(data.getDataStream()), engine, cacheXSLT, loader, data, params, converterHandling, encoding)`
5. `data.setDataStream(resultStream)`; return data.

6. Failure -> `EIException("XSLTProcessor \"{name}\": Unable to execute transformation", cause).`
-

3. Configuration reference

3.1 XSL File

Tag | `XSLTPath` |
Type | XSLT file picker |
Tooltip | The XSLT file to use in transforming XML |

`copyFileAsResource`: when copying a `.xslt` file into route resources, sibling files matching `{basename}--*.xml` are relocated automatically (supporting files pattern).

3.2 Cache XSLT

Tag | `CacheXSLTToXML` |
Default | `false` |
Tooltip | If enabled, the XSLT file will not be refreshed every execution; this can improve performance |

3.3 XSLT Engine

Tag | `XSLTEngine` |
Default | `Saxon - XSLT 3.1` |
Choices | `Xalan (Interpreted) - XSLT 1.0`, `Xalan (Compiled) - XSLT 1.0`, `Saxon - XSLT 3.1` |

Legacy migration (`readConfiguration` / `writeConfiguration`):

Stored legacy	Normalized to
<code>Xalan (Interpreted)</code>	<code>Xalan (Interpreted) - XSLT 1.0</code>
<code>Xalan (Compiled)</code>	<code>Xalan (Compiled) - XSLT 1.0</code>
<code>Saxon</code>	<code>Saxon - XSLT 3.1</code>

3.4 XSLT Parameters

Tag | `XSLTParameters` |
Tab | Advanced |
Type | Table (name / value) |

Names and values evaluated via `ConfigurationManager.getValueFor` at runtime.

3.5 Saxon EchoConverter Handling

Tag | `SaxonConverterHandling` |
Tab | Compatibility |
Default | `Throw Exception` |

Visible when | Engine = **Saxon - XSLT 3.1** |
Choices | Throw Exception, Perform Compatibility Replacement, Skip Converter Check |

3.6 Converter Encoding

Tag | **SaxonConverterEncoding** |
Default | UTF-8 |
Choices | UTF-8, UTF-16, windows-1252 |
Visible when | Saxon + handling is Throw Exception or Perform Compatibility Replacement |

3.7 Inherited AbstractProcessor

Tag | **ExecuteProcessor** |

4. Transaction attributes

None written. Transform parameters may **read** transaction attributes via expression evaluation in the parameters table.

Testing transactions: **XSLTTransformer** checks `com.pilotfish.eip.testingFlag` for test-mode behavior inside the utility class.

5. Worked example

Stylesheet `map-to-csv.xslt`, Engine **Saxon - XSLT 3.1**, parameter:

Name	Value
<code>outputEncoding</code>	UTF-8

Input XML in transaction -> processor replaces stream with transform output (e.g. CSV or alternate XML).

6. Known quirks and caveats

1. **Input stream consumed** — downstream sees only transform output.
2. **Template cache** — stale stylesheet if file changes on disk while cache enabled (except test mode nuances in utility).
3. **Saxon vs Xalan** — XSLT 2/3 features require Saxon.
4. **Xalan ConverterProxy** — Saxon compatibility options address legacy Xalan extension patterns.
5. **InternalErrorListener** class present — warnings/errors logged and rethrown at transformer level inside utility.

7. Operational recommendations

Goal	Guidance
Performance	Enable Cache XSLT for stable production stylesheets
Development	Disable cache while iterating on <code>.xslt</code> files
XSLT 1 legacy	Use Xalan engine; migrate to Saxon when possible
Parameters	Pass dynamic values via expressions referencing transaction attributes
Large XML	Monitor memory; transformer uses streaming/cache patterns inside utility

8. Source index

Topic	Path
XSLT Processor	<code>com/pilotfish/eip/modules/transform/XSLTProcessor</code>
XSLT transformer utility	<code>com/pilotfish/utils/XSLTTransformer.java</code>
Config enums	<code>com/pilotfish/eip/server/XsltConfigTypes.java</code>
Module JAR	<code>xcs-modules-transform-1.0.1.jar</code>

Deep-dive – XSLT Transformation Processor – 26R1.11 – August 2026.